1 Markovian Drone

1.1 Optimal Value Heatmap Plot

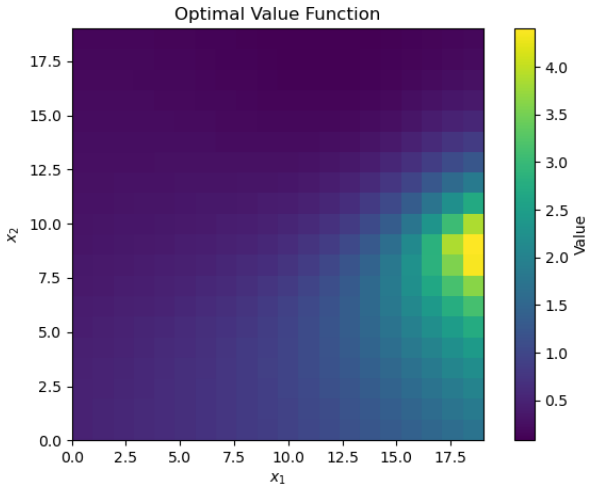


Figure 1: Optimal Value Heatmap Plot

1.2 Optimal Policy Heatmap Plot

As seen in Fig. 2, the simulated drone is following actions depending on the ego-state, denoted by the heatmap. Sometimes the policy is overriden by the storm, that is the reason why the drone is taking sometimes different actions than those of the policy. Also, the simulation runs for 100 steps, so even after reaching the goal, the drone may circle around it.

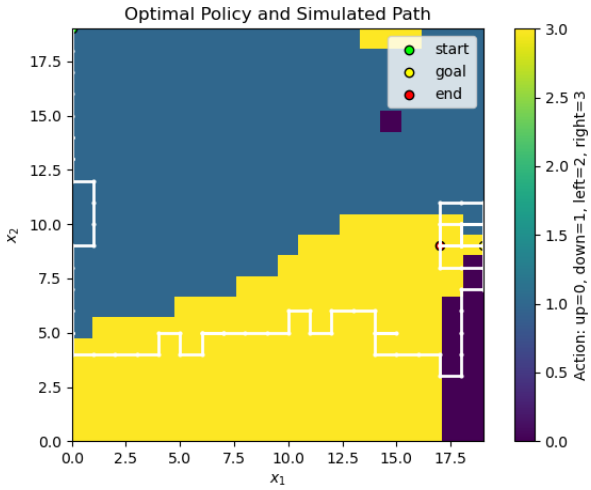


Figure 2: Optimal Policy Heatmap Plot

1.3 Code

Code in Appendix.

2 Reach-avoid flight

(a)

Only the v_y and ω dynamics depend on the thrusts. In the Hamiltonian, the terms involving T_1, T_2 are linear:

$$\left(\frac{p_{v_y}\cos\phi}{m} - \frac{p_\omega \ell}{I_{yy}}\right) T_1 + \left(\frac{p_{v_y}\cos\phi}{m} + \frac{p_\omega \ell}{I_{yy}}\right) T_2$$

So each thrust is chosen at one of its bounds. If the coefficient is negative, use max thrust; otherwise use min thrust.

(b)

For the target set

$$T = [3, 7] \times [-1, 1] \times [-\pi/12, \pi/12] \times [-1, 1]$$

I used

$$h(x) = \max\{3 - y,\ y - 7,\ |v_y| - 1,\ |\phi| - \pi/12,\ |\omega| - 1\}$$

Then $h(x) \leq 0$ exactly when all target constraints are satisfied.

(c)

For the envelope

$$E = [1, 9] \times [-6, 6] \times [-\infty, \infty] \times [-8, 8]$$

I used

$$e(x) = \max\{1 - y,\ y - 9,\ |v_y| - 6,\ |\omega| - 8\}$$

There is no pitch constraint in the envelope, so ϕ does not appear.

(d)

The zero isosurface separates states that can reach the target while staying in the envelope from states that cannot. The tilted ridge/valley comes from the coupling between pitch, angular rate, and vertical acceleration. For example, when the quadrotor is tilted, thrust is not purely vertical, so the same vertical velocity can be recoverable or unrecoverable depending on whether ϕ and ω are rotating the vehicle back toward level flight or farther away from it.

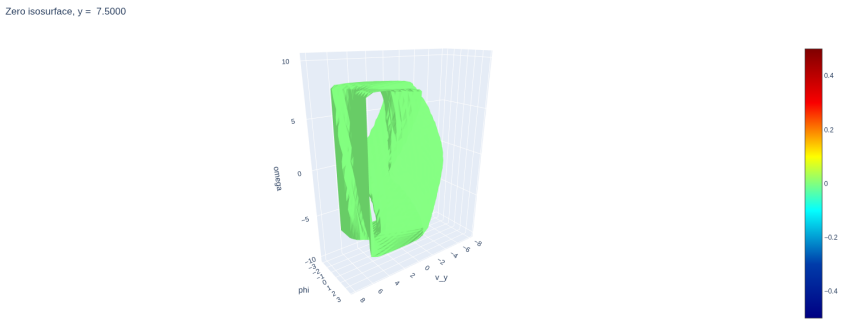


Figure 3: Zero isosurface of the reach-avoid value function

(e)

Dynamic programming gives a global feedback policy over the grid, so online control is cheap once the value function is computed. The downside is that the offline solve is expensive in time and memory, especially as the state dimension increases. MPC is usually easier to adapt online to new constraints or obstacles, but it solves an optimization problem at each time step and is more local.

Code

Code in Appendix.

3 MPC feasibility

3.1 (a) Trajectories

Fig. 4 shows the closed and open loop trajectories for the system. The orange trajectory starts from the larger velocity initial condition, so it is harder to slow down because the input is bounded. Using $P = P_{\mathrm{DARE}}$ gives a more useful terminal cost than $P = I$, since it approximates the infinite-horizon LQR cost-to-go after the short MPC horizon. This makes the predicted plans and the realized trajectory look more stabilizing and less short-sighted.

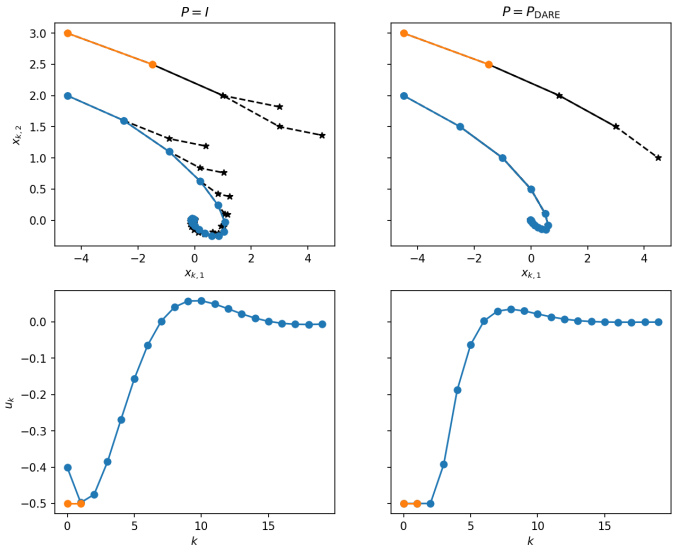


Figure 4: Trajectories

3.2 (b) Region of attraction

Fig. 5 shows the ROA obtained by simulating closed-loop MPC from each grid point and marking the states that converge to the origin. Longer horizons give a larger ROA because the controller can plan farther ahead. The strict terminal constraint $r_f = 0$ makes the ROA smaller, while $r_f = \infty$ leaves feasibility mostly limited by the state and input bounds.

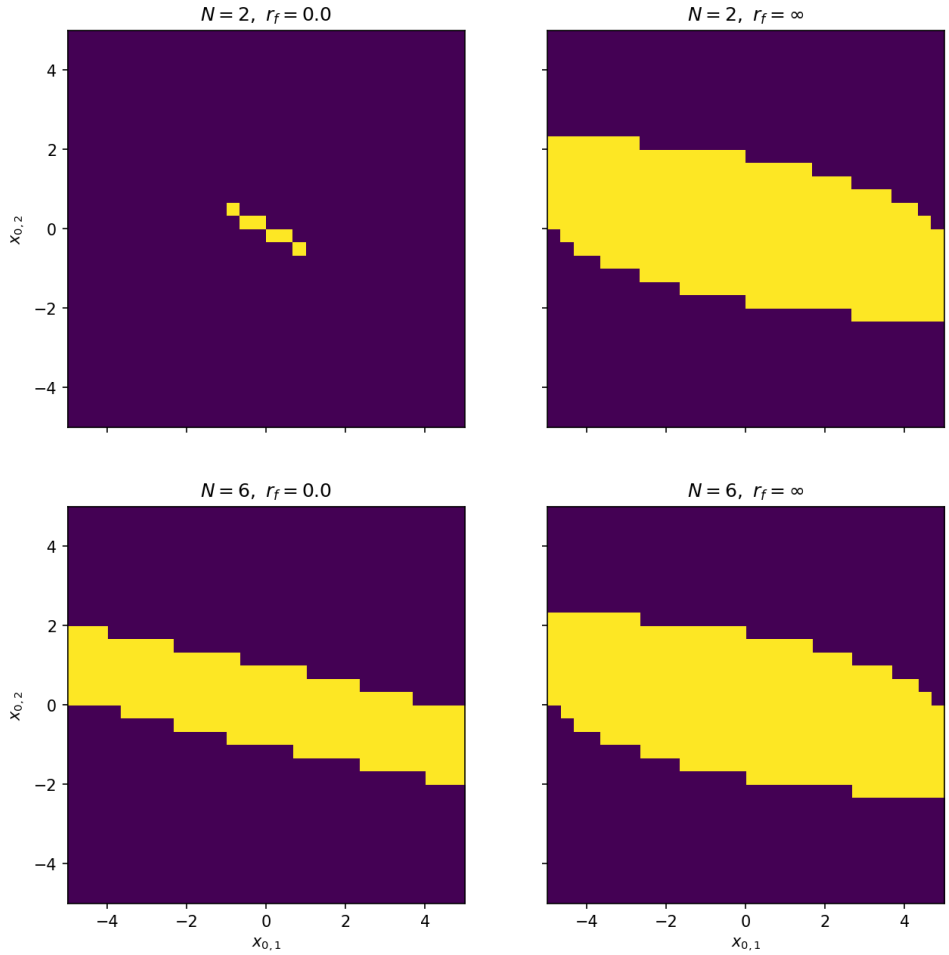


Figure 5: Regions of attraction for different horizon lengths and terminal constraints

3.3 Code

(a)

```
    def do_mpc(
    x0: np.ndarray,
    A: np.ndarray,
    B: np.ndarray,
    P: np.ndarray,
    Q: np.ndarray,
    R: np.ndarray,
    N: int,
    rx: float,
    ru: float,
    rf: float,
) -> tuple[np.ndarray, np.ndarray, str]:
    """Solve the MPC problem starting at state `x0`."""
    n, m = Q.shape[0], R.shape[0]
    x_cvx = cvx.Variable((N + 1, n))
    u_cvx = cvx.Variable((N, m))

    # PART (a): YOUR CODE BELOW ##################################################
    # INSTRUCTIONS: Construct and solve the MPC problem using CVXPY.

    cost = 0.0
```

```
    constraints = [x_cvx[0,:] == x0]

    # N is the horizon
    for t in range(N):
        # running cost
        cost += cvx.quad_form(x_cvx[t,:], Q) + cvx.quad_form(u_cvx[t,:], R)

        constraints.append(x_cvx[t+1,:] == A@x_cvx[t,:] + B@u_cvx[t,:])
        constraints.append(cvx.norm_inf(x_cvx[t,:]) <= rx)
        constraints.append(cvx.norm_inf(u_cvx[t,:]) <= ru)

    # auxiliary cost
    cost += cvx.quad_form(x_cvx[N,:], P)
    constraints.append(cvx.norm_inf(x_cvx[N,:]) <= rx)
    constraints.append(cvx.norm_inf(x_cvx[N,:]) <= rf)

    # END PART (a) #############################################################

    prob = cvx.Problem(cvx.Minimize(cost), constraints)
    prob.solve(cvx.CLARABEL)
    x = x_cvx.value
    u = u_cvx.value
    status = prob.status

    return x, u, status
```

(b)

```
    def compute_roa(
    A: np.ndarray,
    B: np.ndarray,
    P: np.ndarray,
    Q: np.ndarray,
    R: np.ndarray,
    N: int,
    rx: float,
    ru: float,
    rf: float,
    grid_dim: int = 21,
    max_steps: int = 20,
    tol: float = 1e-2,
) -> np.ndarray:
    """Compute a region of attraction."""
    roa = np.zeros((grid_dim, grid_dim))
    xs = np.linspace(-rx, rx, grid_dim)
    for i, x1 in enumerate(xs):
        for j, x2 in enumerate(xs):
            x = np.array([x1, x2])
            # PART (b): YOUR CODE BELOW ########################################
            # INSTRUCTIONS: Simulate the closed-loop system for `max_steps`,
            #               stopping early only if the problem becomes
            #               infeasible or the state has converged close enough
            #               to the origin. If the state converges, flag the
            #               corresponding entry of `roa` with a value of `1`.

            for _ in range(max_steps):
                x_mpc, u_mpc, status = do_mpc(x, A, B, P, Q, R, N, rx, ru, rf)

                if status == "infeasible":
                    break

```

```
                if np.linalg.norm(x) <= tol:
                    roa[i, j] = 1
                    break

                x = A @ x + B @ u_mpc[0, :]

        if np.linalg.norm(x) <= tol:
            roa[i, j] = 1

            # END PART (b) ########################################################
    return roa
```

4 Terminal ingredients

(a)

Eigenvalues of A are $(0.9, 0.8)$, and lie in the unit circle, so A is asymptotically stable, and $u = 0$ is feasible as terminal controller and the terminal ingredients are designed for the autonomous system $x_{t+1} = Ax_t$. The terminal set X_T should satisfy $X_T \subseteq X$ and be a positive invariant for the prev. system:

$$x \in X_T \implies Ax \in X_T.$$

which guarantees recursive feasibility by shifting the MPC solution and appending $u = 0$.

$P \succ 0$ a symmetric, positive definite matrix

For stability, P must a symmetric, positive definite matrix ($P \succ 0$), from the Lyapunov equation.

(b)

For any $x \in \mathcal{X}_T$ (positive invariant set), $x^\top W x \leq 1$.

Since $A^\top W A - W \preceq 0$, we get:

$$\begin{aligned} x^\top (A^\top W A - W) x &\leq 0 \\ (Ax)^\top W A x - x^\top W x &\leq 0 \\ (Ax)^\top W A x \leq x^\top W x &\leq 1 \end{aligned}$$

So $Ax_T \in \mathcal{X}_T$

For $I - r_x^2 W \preceq 0$:

$$\begin{aligned} x^\top (I - r_x^2 W) x &\leq 0 \\ x^\top x - x^\top r_x^2 W x &\leq 0 \\ x^\top x &\leq x^\top r_x^2 W x \\ x^\top x / r_x^2 \leq x^\top W x &\leq 1 \\ x^\top x &\leq r_x^2 \end{aligned}$$

Which already complies with $x^\top x \leq \|x\|_2^2 \leq r_x^2$

(c)

First Constraint

Using the first hint

$$\begin{aligned} A^\top M^{-1} A - M^{-1} &\preceq 0 \\ M(A^\top M^{-1} A) M - M M^{-1} M &\preceq 0 \\ M A^\top M^{-1} A M - M &\preceq 0 \end{aligned}$$

Using Schur's complement lemma, the constraint is rewritten as:

$$\begin{bmatrix} M & AM \\ MA^T & M \end{bmatrix} \succeq 0$$

Second Constraint

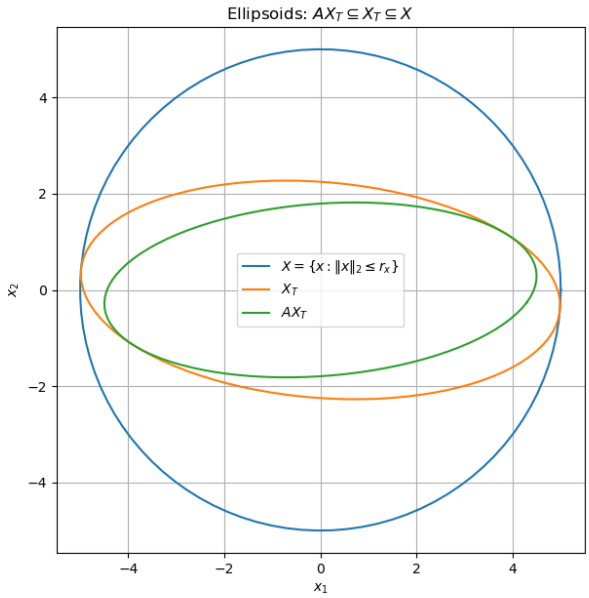
$$\begin{aligned} I - r_x^2 M^{-1} &\preceq 0 \\ r_x^{-2} I &\preceq M^{-1} \end{aligned}$$

Using second hint

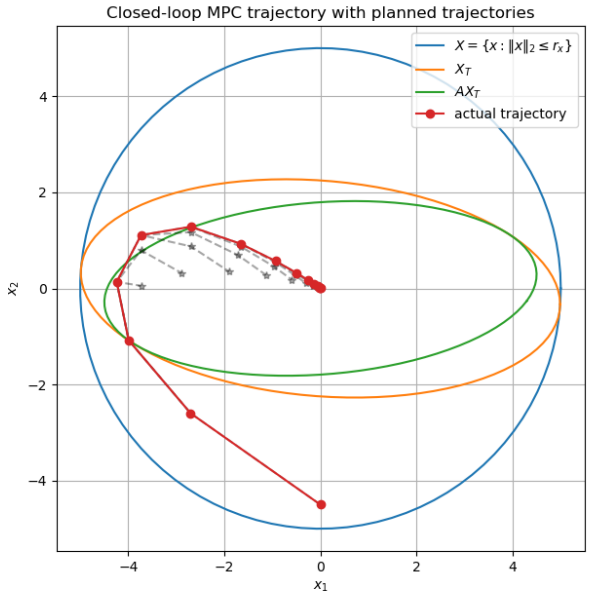
$$\begin{aligned} r_x^2 I &\succeq M \\ r_x^2 I - M &\succeq 0 \end{aligned}$$

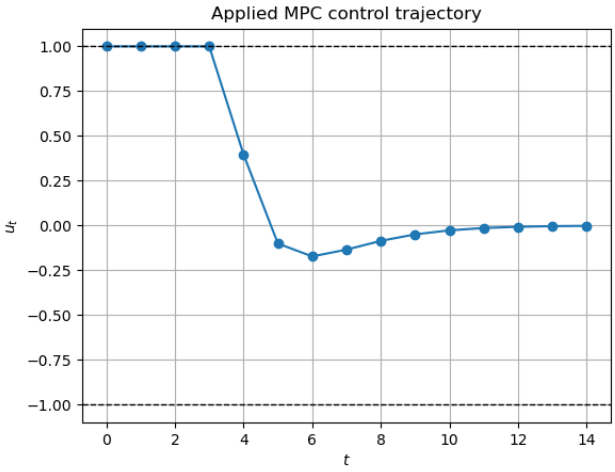
(d)

```
W=
[[0.041␣0.013]
␣[0.013␣0.198]]
```



(d)





Code

Code in Appendix.

Code Appendix

Homework part	Code file
3.1	markovian_drone.py
3.2	starter_hj_reachability.py
3.4	terminal_ingredients.py

markovian_drone.py

```python
import numpy as np
import matplotlib.pyplot as plt

ACTIONS = np.array([  [0,1],  # up
                      [0,-1], # down
                      [-1,0], # left
                      [1,0],  # right
                   ])

def idx_to_state(s,n=20):
    x1 = s % n
    x2 = s // n
    return np.array([x1,x2])

def move(x,a,n=20):
    x_next = x+a
    x_next = np.clip(x_next,0,n-1)

    return x_next[1]*n + x_next[0]

def value_iteration(n,eps=1e-6):
    sigma = 10
    gamma = 0.95
    x_eye = np.array([15,15])
    x_goal = np.array([19,9])
    num_states = n*n
    V = np.zeros(n*n)
    opt_policy = np.zeros(n*n, dtype=int)

    T = np.zeros((n*n,4,n*n))   # current, action, next
    R = np.zeros(num_states)
    R[x_goal[1] * n + x_goal[0]] = 1

    for s in range(num_states):
        # storm_influence
        x = idx_to_state(s)
        omega = np.exp(-np.linalg.norm(x - x_eye)**2 / (2 * sigma**2))

        # Build transition
        for a in range(4):
            # specified action
            next_s = move(x,ACTIONS[a])
            T[s,a, next_s] += 1 - omega

            # random action
            for a_rand in range(4):
                next_s_rand = move(x,ACTIONS[a_rand])
                T[s,a,next_s_rand] += omega/4

    diff = np.inf
    while diff > eps:
```

```
        V_new = np.zeros_like(V)

        for s in range(num_states):
            q = np.zeros(4)

            for a in range(4):
                q[a] =  np.sum(T[s,a,:] * (R + gamma*V))

            opt_policy[s] = np.argmax(q)
            V_new[s] = q[opt_policy[s]]#np.max(q)

        diff = np.max(np.abs(V_new - V))
        V = V_new

    return V, opt_policy

def simulate_MDP(policy,n):
    sigma = 10
    x_eye = np.array([15,15])
    x_init = np.array([0,19])
    time_steps = 100
    path = [x_init.copy()]

    x = x_init.copy()
    for _ in range(time_steps):
        s = x[1] * n + x[0]
        action = policy[s]
        omega = np.exp(-np.linalg.norm(x - x_eye)**2 / (2 * sigma**2))

        if np.random.rand() < omega:
            action = np.random.randint(4)

        next_s = move(x,ACTIONS[action],n)
        x = idx_to_state(next_s,n)
        path.append(x.copy())

    return np.array(path)

def plot_heatmaps(V,policy,path,n):
    V_grid = V.reshape((n,n))
    policy_grid = policy.reshape((n,n))

    plt.figure()
    plt.imshow(V_grid, origin='lower', extent=[0,n-1,0,n-1])
    plt.colorbar(label='Value')
    plt.xlabel('$x_1$')
    plt.ylabel('$x_2$')
    plt.title('Optimal Value Function')
    plt.tight_layout()
    # plt.show()
    plt.savefig("optimal_value_heatmap.png")

    plt.figure()
    plt.imshow(policy_grid, origin='lower', extent=[0,n-1,0,n-1], vmin=0, vmax=3)
    plt.colorbar(label='Action: up=0, down=1, left=2, right=3')
    plt.plot(path[:,0], path[:,1], color='white', linewidth=2, marker='o', markersize=2)
    plt.scatter(path[0,0], path[0,1], color='lime', edgecolor='black', label='start')
    plt.scatter(19,9, color='yellow', edgecolor='black', label='goal')
    plt.scatter(path[-1,0], path[-1,1], color='red', edgecolor='black', label='end')
    plt.xlabel('$x_1$')
    plt.ylabel('$x_2$')
    plt.title('Optimal Policy and Simulated Path')
    plt.legend()
```

```
    plt.tight_layout()
    plt.savefig("policy_heatmap_path.png")

def main():
    n = 20

    # Part A
    V,policy = value_iteration(n)

    # Part B
    path = simulate_MDP(policy,n)

    plot_heatmaps(V,policy,path,n)


if __name__ == '__main__':
    main()
```

starter_hj_reachability.py

```
"""
Starter code for the problem "Hamilton-Jacobi reachability".

Autonomous Systems Lab (ASL), Stanford University
"""

import os

import jax
import jax.numpy as jnp
import numpy as np

import hj_reachability as hj

import matplotlib.pyplot as plt
from animations import animate_planar_quad

# Define problem ingredients (exercise parts (a), (b), (c)).


class PlanarQuadrotor:

    def __init__(self):
        # Dynamics constants
        # yapf: disable
        self.g = 9.807          # gravity (m / s**2)
        self.m = 2.5            # mass (kg)
        self.l = 1.0            # half-length (m)
        self.Iyy = 1.0          # moment of inertia about the out-of-plane axis (kg * m**2)
        self.Cd_v = 0.25        # translational drag coefficient
        self.Cd_phi = 0.02255   # rotational drag coefficient
        # yapf: enable

        # Control constraints
        self.max_thrust_per_prop = (
            0.75 * self.m * self.g
        )  # total thrust-to-weight ratio = 1.5
        self.min_thrust_per_prop = (
            0  # at least until variable-pitch quadrotors become mainstream :D
        )

```

```
    def full_dynamics(self, full_state, control):
        """Continuous-time␣dynamics␣of␣a␣planar␣quadrotor␣expressed␣as␣an␣ODE."""
        x, v_x, y, v_y, phi, omega = full_state
        T_1, T_2 = control
        return jnp.array(
            [
                v_x,
                (-(T_1 + T_2) * jnp.sin(phi) - self.Cd_v * v_x) / self.m,
                v_y,
                ((T_1 + T_2) * jnp.cos(phi) - self.Cd_v * v_y) / self.m - self.g,
                omega,
                ((T_2 - T_1) * self.l - self.Cd_phi * omega) / self.Iyy,
            ]
        )

    def dynamics(self, state, control):
        """Reduced␣(for␣the␣purpose␣of␣reachable␣set␣computation)␣continuous-time␣dynamics
            ␣of␣a␣planar␣quadrotor."""
        y, v_y, phi, omega = state
        T_1, T_2 = control
        return jnp.array(
            [
                v_y,
                ((T_1 + T_2) * jnp.cos(phi) - self.Cd_v * v_y) / self.m - self.g,
                omega,
                ((T_2 - T_1) * self.l - self.Cd_phi * omega) / self.Iyy,
            ]
        )

    def optimal_control(self, state, grad_value):
        """Computes␣the␣optimal␣control␣realized␣by␣the␣HJ␣PDE␣Hamiltonian.

␣␣␣␣␣␣␣␣Args:
␣␣␣␣␣␣␣␣␣␣␣␣state:␣An␣unbatched␣(!)␣state␣vector,␣an␣array␣of␣shape␣`(4,)`␣containing␣`[y,
    ␣v_y,␣phi,␣omega]`.
␣␣␣␣␣␣␣␣␣␣␣␣grad_value:␣An␣array␣of␣shape␣`(4,)`␣containing␣the␣gradient␣of␣the␣value␣
    function␣at␣`state`.

␣␣␣␣␣␣␣␣Returns:
␣␣␣␣␣␣␣␣␣␣␣␣A␣vector␣of␣optimal␣controls,␣an␣array␣of␣shape␣`(2,)`␣containing␣`[T_1,␣T_2
    ]`,␣that␣minimizes
␣␣␣␣␣␣␣␣␣␣␣␣`grad_value␣@␣self.dynamics(state,␣control)`.
␣␣␣␣␣␣␣␣"""
        # PART (a): WRITE YOUR CODE BELOW ###################################################
        # You may find `jnp.where` to be useful; see corresponding numpy docstring:
        # https://numpy.org/doc/stable/reference/generated/numpy.where.html
        y, v_y, phi, omega = state
        p_y, p_v_y, p_phi, p_omega = grad_value
        del y, v_y, p_y, p_phi

        thrust_min = jnp.asarray(self.min_thrust_per_prop, dtype=jnp.float32)
        thrust_max = jnp.asarray(self.max_thrust_per_prop, dtype=jnp.float32)

        coeff_T_1 = p_v_y * jnp.cos(phi) / self.m - p_omega * self.l / self.Iyy
        coeff_T_2 = p_v_y * jnp.cos(phi) / self.m + p_omega * self.l / self.Iyy

        T_1 = jnp.where(coeff_T_1 <= 0.0, thrust_max, thrust_min)
        T_2 = jnp.where(coeff_T_2 <= 0.0, thrust_max, thrust_min)
        return jnp.array([T_1, T_2])
        ###################################################################################

    def hamiltonian(self, state, time, value, grad_value):
        """Evaluates␣the␣HJ␣PDE␣Hamiltonian."""
```

```
        del time, value  # unused
        control = self.optimal_control(state, grad_value)
        return grad_value @ self.dynamics(state, control)

    def partial_max_magnitudes(self, state, time, value, grad_value_box):
        """Computes␣the␣max␣magnitudes␣of␣the␣Hamiltonian␣partials␣over␣the␣`
            grad_value_box`␣in␣each␣dimension."""
        del time, value, grad_value_box  # unused
        y, v_y, phi, omega = state
        return jnp.array(
            [
                jnp.abs(v_y),
                (
                    2 * self.max_thrust_per_prop * jnp.abs(jnp.cos(phi))
                    + self.Cd_v * jnp.abs(v_y)
                )
                / self.m
                + self.g,
                jnp.abs(omega),
                (
                    (self.max_thrust_per_prop - self.min_thrust_per_prop) * self.l
                    + self.Cd_phi * jnp.abs(omega)
                )
                / self.Iyy,
            ]
        )


def target_set(state):
    """A␣real-valued␣function␣such␣that␣the␣zero-sublevel␣set␣is␣the␣target␣set.

␣␣␣␣Args:
␣␣␣␣␣␣␣␣state:␣An␣unbatched␣(!)␣state␣vector,␣an␣array␣of␣shape␣`(4,)`␣containing␣`[y,␣v_y
    ,␣phi,␣omega]`.

␣␣␣␣Returns:
␣␣␣␣␣␣␣␣A␣scalar,␣nonpositive␣iff␣the␣state␣is␣in␣the␣target␣set.
␣␣␣␣"""
    # PART (b): WRITE YOUR CODE BELOW ################################################
    y, v_y, phi, omega = state
    return jnp.max(
        jnp.array(
            [
                3.0 - y,
                y - 7.0,
                jnp.abs(v_y) - 1.0,
                jnp.abs(phi) - jnp.pi / 12.0,
                jnp.abs(omega) - 1.0,
            ]
        )
    )
    ##################################################################################


def envelope_set(state):
    """A␣real-valued␣function␣such␣that␣the␣zero-sublevel␣set␣is␣the␣operational␣envelope.

␣␣␣␣Args:
␣␣␣␣␣␣␣␣state:␣An␣unbatched␣(!)␣state␣vector,␣an␣array␣of␣shape␣`(4,)`␣containing␣`[y,␣v_y
    ,␣phi,␣omega]`.

␣␣␣␣Returns:
␣␣␣␣␣␣␣␣A␣scalar,␣nonpositive␣iff␣the␣state␣is␣in␣the␣operational␣envelope.
```

```
␣␣␣␣"""
    # PART (c): WRITE YOUR CODE BELOW ################################################
    y, v_y, phi, omega = state
    return jnp.max(
        jnp.array(
            [
                1.0 - y,
                y - 9.0,
                jnp.abs(v_y) - 6.0,
                jnp.abs(omega) - 8.0,
            ]
        )
    )
    ################################################################################


def test_optimal_control(n=10, seed=0):
    planar_quadrotor = PlanarQuadrotor()
    optimal_control = jax.jit(planar_quadrotor.optimal_control)
    np.random.seed(seed)
    states = 5 * np.random.normal(size=(n, 4))
    grad_values = np.random.normal(size=(n, 4))
    try:
        for state, grad_value in zip(states, grad_values):
            if not jnp.issubdtype(
                optimal_control(state, grad_value).dtype, jnp.floating
            ):
                raise ValueError(
                    "`PlanarQuadrotor.optimal_control`␣must␣return␣a␣`float`␣array␣(i.e.,␣
                        not␣`int`)."
                )
            opt_hamiltonian_value = grad_value @ planar_quadrotor.dynamics(
                state, optimal_control(state, grad_value)
            )
            for T_1 in (
                planar_quadrotor.min_thrust_per_prop,
                planar_quadrotor.max_thrust_per_prop,
            ):
                for T_2 in (
                    planar_quadrotor.min_thrust_per_prop,
                    planar_quadrotor.max_thrust_per_prop,
                ):
                    hamiltonian_value = grad_value @ planar_quadrotor.dynamics(
                        state, np.array([T_1, T_2])
                    )
                    if opt_hamiltonian_value > hamiltonian_value + 1e-4:
                        raise ValueError(
                            "Check␣your␣logic␣for␣`PlanarQuadrotor.optimal_control`;␣with␣
                                "
                            f"`state`␣{state}␣and␣`grad_value`␣{grad_value},␣got␣optimal␣
                                control"
                            f"{optimal_control(state,␣grad_value)}␣with␣corresponding␣
                                Hamiltonian␣value␣"
                            f"{opt_hamiltonian_value:7.4f}␣but␣{np.array([T_1,␣T_2])}␣has␣
                                a␣lower␣corresponding␣"
                            f"value␣{hamiltonian_value:7.4f}."
                        )
    except (jax.errors.JAXTypeError, jax.errors.JAXIndexError, AssertionError) as e:
        print(
            "`PlanarQuadrotor.optimal_control`␣must␣be␣implemented␣using␣only␣`jnp`␣
                operations;␣"
            "`np`␣may␣only␣be␣used␣for␣constants,␣"
```

```
            "and␣`jnp.where`␣must␣be␣used␣instead␣of␣native␣python␣control␣flow␣(`if`/`else`)."
        )
        raise e


def test_target_set():
    try:
        in_states = [
            np.array([5.0, 0.0, 0.0, 0.0]),
            np.array([6.0, 0.1, 0.1, 0.1]),
            # feel free to add test cases
        ]
        out_states = [
            np.array([2.0, 0.0, 0.0, 0.0]),
            np.array([5.0, 2.0, 0.0, 0.0]),
            np.array([5.0, 0.0, 2.0, 0.0]),
            np.array([5.0, 0.0, 0.0, 2.0]),
            # feel free to add test cases
        ]
        for x in in_states:
            if not jnp.issubdtype(target_set(x).dtype, jnp.floating):
                raise ValueError(
                    "`target_set`␣must␣return␣a␣`float`␣scalar␣(i.e.,␣not␣`int`)."
                )
            if target_set(x) > 0:
                raise ValueError(
                    f"Check␣your␣logic␣for␣`target_set`;␣for␣`state`␣{x}␣(in)␣you␣have␣target_set(state)␣=␣"
                    f"{target_set(x)}."
                )
        for x in out_states:
            if target_set(x) <= 0:
                raise ValueError(
                    f"Check␣your␣logic␣for␣`target_set`;␣for␣`state`␣{x}␣(out)␣you␣have␣target_set(state)␣=␣"
                    f"{target_set(x)}."
                )
    except (jax.errors.JAXTypeError, jax.errors.JAXIndexError, AssertionError) as e:
        print(
            "`target_set`␣must␣be␣implemented␣using␣only␣`jnp`␣operations,␣"
            "`np`␣may␣only␣be␣used␣for␣constants,␣"
            "and␣`jnp.where`␣must␣be␣used␣instead␣of␣native␣python␣control␣flow␣(`if`/`else`)."
        )
        raise e


def test_envelope_set():
    try:
        in_states = [
            np.array([5.0, 0.0, 0.0, 0.0]),
            np.array([7.0, 5.0, 100.0, 6.0]),
            # feel free to add test cases
        ]
        out_states = [
            np.array([0.0, 0.0, 0.0, 0.0]),
            np.array([5.0, 8.0, 0.0, 0.0]),
            np.array([5.0, 0.0, 0.0, 10.0]),
            # feel free to add test cases
        ]
        for x in in_states:
            if not jnp.issubdtype(envelope_set(x).dtype, jnp.floating):
```

```
                raise ValueError(
                    "`envelope_set` must return a `float` scalar (i.e., not `int`)."
                )
            if envelope_set(x) > 0:
                raise ValueError(
                    f"Check your logic for `envelope_set`; for `state` {x} (in) you have
                        envelope_set(state) = "
                    f"{envelope_set(x)}."
                )
        for x in out_states:
            if envelope_set(x) <= 0:
                raise ValueError(
                    f"Check your logic for `envelope_set`; for `state` {x} (out) you have
                        envelope_set(state) = "
                    f"{envelope_set(x)}."
                )
    except (jax.errors.JAXTypeError, jax.errors.JAXIndexError, AssertionError) as e:
        print(
            "`envelope_set` must be implemented using only `jnp` operations, "
            "`np` may only be used for constants, "
            "and `jnp.where` must be used instead of native python control flow (`if`/`
                else`)."
        )
        raise e


test_optimal_control()
test_target_set()
test_envelope_set()

# Set up problem for use with PDE solver.
planar_quadrotor = PlanarQuadrotor()
state_domain = hj.sets.Box(
    lo=np.array([0.0, -8.0, -np.pi, -10.0]), hi=np.array([10.0, 8.0, np.pi, 10.0])
)
grid_resolution = (
    25,
    25,
    30,
    25,
)  # can/should be increased if running on GPU, or if extra patient
grid = hj.Grid.from_lattice_parameters_and_boundary_conditions(
    state_domain, grid_resolution, periodic_dims=2
)

target_values = hj.utils.multivmap(target_set, np.arange(4))(grid.states)
envelope_values = hj.utils.multivmap(envelope_set, np.arange(4))(grid.states)
terminal_values = np.maximum(target_values, envelope_values)

solver_settings = hj.SolverSettings.with_accuracy(
    "medium",  # can/should be changed to "very_high" if running on GPU, or if extra
        patient
    hamiltonian_postprocessor=lambda x: jnp.minimum(x, 0),
    value_postprocessor=lambda t, x: jnp.maximum(x, envelope_values),
)

# Propagate the HJ PDE _backwards_ in time.
initial_time = 0.0
final_time = -5.0
yn = None
if os.path.exists("hj_reachability_values.npz"):
    yn = (
        input(
```

```
            "Existing␣hj_reachability_values.npz␣file␣found␣from␣a␣previous␣solve;␣use␣it␣(Y/n)?␣"
        )
        .lower()
        .strip()
    )
if yn is None or yn == "n":
    print("Computing␣the␣value␣function␣by␣solving␣the␣HJ␣PDE.")
    values = hj.step(
        solver_settings,
        planar_quadrotor,
        grid,
        initial_time,
        terminal_values,
        final_time,
    ).block_until_ready()
    print("Saving␣the␣value␣function␣to␣hj_reachability_values.npz.")
    np.savez("hj_reachability_values.npz", values=values)
else:
    print("Loading␣previously␣computed␣value␣function␣from␣hj_reachability_values.npz.")
    values = np.load("hj_reachability_values.npz")["values"]
grad_values = grid.grad_values(values)

# Utilities for rolling out the optimal controls and visualizing.


@jax.jit
def optimal_step(full_state, dt):
    state = full_state[2:]
    grad_value = grid.interpolate(grad_values, state)
    control = planar_quadrotor.optimal_control(state, grad_value)
    return full_state + dt * planar_quadrotor.full_dynamics(full_state, control)


def optimal_trajectory(full_state, dt=1 / 100, T=5):
    full_states = [full_state]
    t = np.arange(T / dt) * dt
    for _ in t:
        full_states.append(optimal_step(full_states[-1], dt))
    return t, np.array(full_states)


def animate_optimal_trajectory(full_state, dt=1 / 100, T=5):
    t, full_states = optimal_trajectory(full_state, dt, T)
    value = grid.interpolate(values, full_state[2:])
    fig, anim = animate_planar_quad(
        t,
        full_states[:, 0],
        full_states[:, 2],
        full_states[:, 4],
    )
    return fig, anim


# Dropping the quad straight down (v_y = -5, mimicking waiting for a sec after the drop to turn the props on).
state = [5.0, -5.0, 0.0, 0.0]
fig, ani = animate_optimal_trajectory(np.array([0, 0] + state))
ani.save("planar_quad_1.mp4", writer="ffmpeg")
# plt.show()

# Flipping the quad up into the air.
state = [6.0, 2.0, -3 * np.pi / 4, -4.0]
```

```
fig, ani = animate_optimal_trajectory(np.array([0, 0] + state))
ani.save("planar_quad_2.mp4", writer="ffmpeg")
# plt.show()

# Dropping the quad like a falling leaf.
state = [8.0, -0.8, np.pi / 2, 2.0]
fig, ani = animate_optimal_trajectory(np.array([0, 0] + state))
ani.save("planar_quad_3.mp4", writer="ffmpeg")
# plt.show()

# Too much negative vertical velocity to recover before hitting the floor.
state = [8.0, -3.0, np.pi / 2, 2.0]
fig, ani = animate_optimal_trajectory(np.array([0, 0] + state))
ani.save("planar_quad_4.mp4", writer="ffmpeg")
# plt.show()

# Examining an isosurface (exercise part (d)).
import plotly.graph_objects as go

i_y = 18
fig = go.Figure(
    data=go.Isosurface(
        x=grid.states[i_y, ..., 1].ravel(),
        y=grid.states[i_y, ..., 2].ravel(),
        z=grid.states[i_y, ..., 3].ravel(),
        value=values[i_y].ravel(),
        colorscale="jet",
        isomin=0,
        surface_count=1,
        isomax=0,
    ),
    layout_title=f"Zero␣isosurface,␣y␣=␣{grid.coordinate_vectors[0][i_y]:7.4f}",
    layout_scene_xaxis_title="v_y",
    layout_scene_yaxis_title="phi",
    layout_scene_zaxis_title="omega",
)
fig.show()
```

terminal_ingredients.py

```
import numpy as np
import matplotlib.pyplot as plt

import cvxpy as cp

def generate_ellipsoid_points(M, num_points=100):
    """Generate␣points␣on␣a␣2-D␣ellipsoid.
␣␣␣␣The␣ellipsoid␣is␣described␣by␣the␣equation
␣␣␣␣‘{␣x␣|␣x.T␣@␣inv(M)␣@␣x␣<=␣1␣}‘,
␣␣␣␣where␣‘inv(M)‘␣denotes␣the␣inverse␣of␣the␣matrix␣argument␣‘M‘.
␣␣␣␣The␣returned␣array␣has␣shape␣(num_points,␣2).
␣␣␣␣"""
    L = np.linalg.cholesky(M)
    θ = np.linspace(0, 2*np.pi, num_points)
    u = np.column_stack([np.cos(θ), np.sin(θ)])
    x = u @ L.T
    return x

def semi_def_program():
    A = np.array([
        [0.9, 0.6],
```

```
        [0.0, 0.8]
    ])
    B = np.array([
        [0.0],
        [1.0]
    ])

    rx = 5.0
    n = A.shape[0]
    M = cp.Variable((n, n), symmetric=True)

    block = cp.bmat([
        [M, A @ M],
        [M @ A.T, M]
    ])

    constraints = [
        M >> 0,
        block >> 0,
        rx**2 * np.eye(n) - M >> 0
    ]

    objective = cp.Maximize(cp.log_det(M))
    prob = cp.Problem(objective,constraints)

    prob.solve()

    # print(M.value)
    W = np.linalg.inv(M.value)
    print("W=")
    print(np.round(W, 3))

    X_T = generate_ellipsoid_points(M.value)

    X_T_next = X_T @ A.T

    # X = {x | ||x||_2 <= rx}
    theta = np.linspace(0, 2*np.pi, 100)
    X_pts = np.column_stack([
        rx * np.cos(theta),
        rx * np.sin(theta)
    ])

    plt.figure(figsize=(7, 7))

    plt.plot(X_pts[:, 0], X_pts[:, 1], label=r"$X=\{x:\|x\|_2 \leq r_x\}$")
    plt.plot(X_T[:, 0], X_T[:, 1], label=r"$X_T$")
    plt.plot(X_T_next[:, 0], X_T_next[:, 1], label=r"$A X_T$")

    plt.axis("equal")
    plt.grid(True)
    plt.xlabel(r"$x_1$")
    plt.ylabel(r"$x_2$")
    plt.legend()
    plt.title(r"Ellipsoids: $A X_T \subseteq X_T \subseteq X$")
    plt.savefig("ellipsoids.png")

    return A, B, M.value, W

def setup_mpc(A, B, W, N=4, rx=5.0, ru=1.0):
    n, m = B.shape
    Q = np.eye(n)
    R = np.eye(m)
```

```
    P = np.eye(n)

    x0 = cp.Parameter(n)
    x = cp.Variable((N + 1, n))
    u = cp.Variable((N, m))

    cost = 0.0
    constraints = [x[0, :] == x0]

    for t in range(N):
        cost += cp.quad_form(x[t, :], Q) + cp.quad_form(u[t, :], R)
        constraints += [
            x[t + 1, :] == A @ x[t, :] + B @ u[t, :],
            cp.norm(x[t, :], 2) <= rx,
            cp.norm(u[t, :], 2) <= ru,
        ]

    cost += cp.quad_form(x[N, :], P)
    constraints += [cp.quad_form(x[N, :], W) <= 1]

    prob = cp.Problem(cp.Minimize(cost), constraints)
    return prob, x0, x, u

def simulate_mpc(A, B, M, W, N=4, T=15, rx=5.0):
    prob, x0_param, x_var, u_var = setup_mpc(A, B, W, N=N, rx=rx)

    x = np.array([0.0, -4.5])
    actual_states = [x.copy()]
    planned_states = []
    applied_controls = []

    for _ in range(T):
        x0_param.value = x
        prob.solve()

        x_plan = x_var.value
        u_plan = u_var.value
        planned_states.append(x_plan.copy())
        applied_controls.append(u_plan[0, 0])

        x = A @ x + B @ u_plan[0, :]
        actual_states.append(x.copy())

    actual_states = np.array(actual_states)
    applied_controls = np.array(applied_controls)

    plot_mpc_trajectories(A, M, actual_states, planned_states, rx)
    plot_control_trajectory(applied_controls)

def plot_mpc_trajectories(A, M, actual_states, planned_states, rx):
    X_T = generate_ellipsoid_points(M)
    X_T_next = X_T @ A.T

    theta = np.linspace(0, 2*np.pi, 100)
    X_pts = np.column_stack([
        rx * np.cos(theta),
        rx * np.sin(theta)
    ])

    plt.figure(figsize=(7, 7))
    plt.plot(X_pts[:, 0], X_pts[:, 1], label=r"$X=\{x:\|x\|_2␣\leq␣r_x\}$")
    plt.plot(X_T[:, 0], X_T[:, 1], label=r"$X_T$")
    plt.plot(X_T_next[:, 0], X_T_next[:, 1], label=r"$A␣X_T$")
```

```
148
149     for x_plan in planned_states:
150         plt.plot(x_plan[:, 0], x_plan[:, 1], "--*", color="k", alpha=0.35)
151
152     plt.plot(actual_states[:, 0], actual_states[:, 1], "-o", label="actual trajectory")
153     plt.axis("equal")
154     plt.grid(True)
155     plt.xlabel(r"$x_1$")
156     plt.ylabel(r"$x_2$")
157     plt.legend()
158     plt.title("Closed-loop MPC trajectory with planned trajectories")
159     plt.savefig("terminal_mpc_trajectory.png")
160
161 def plot_control_trajectory(applied_controls):
162     plt.figure()
163     plt.plot(np.arange(len(applied_controls)), applied_controls, "-o")
164     plt.axhline(1.0, color="k", linestyle="--", linewidth=1)
165     plt.axhline(-1.0, color="k", linestyle="--", linewidth=1)
166     plt.grid(True)
167     plt.xlabel(r"$t$")
168     plt.ylabel(r"$u_t$")
169     plt.title("Applied MPC control trajectory")
170     plt.savefig("terminal_mpc_control.png")
171
172 def main():
173     A, B, M, W = semi_def_program()
174     simulate_mpc(A, B, M, W)
175
176 if __name__ == "__main__":
177     main()
```